

Assignment 3

Julia Nething
nething@gatech.edu

1 RESEARCH LOG

1.1 Background

Over the past two assignments, I have been looking into the most effective ways to teach difficult topics such as recursion. I teach at a software engineering immersive (Hack Reactor) and recursion in particular has been a very difficult topic to help students with, so I would like to focus my project on developing either curriculum or a tool to help students learn recursion.

In the first assignment, I focused solely on different strategies for teaching recursion to students. In the second assignment, I branched out a bit to understand how similar topics like dynamic programming are taught and how more broad-reaching topics such as debugging techniques are addressed. I also looked into more meta topics such as how to design effective pre and post tests and how to choose an effective order of material (such as using the interleaving strategy).

This week, I would like to look into blended learning and flipped classroom models and see how that affects programming education, thanks to advice from my peer feedback. Additionally, I would like to look into how to give effective feedback to students, especially immediate useful feedback. Finally, I would like to see if I can find more examples of how professors use visualizations in computer science education.

1.2 Papers

1.2.1 *The Effectiveness of the Blended Learning Approach in Algorithm and Programming Courses*

IDMBA Darmawan et al. (2021). "The effectiveness of the blended learning approach in algorithm and programming courses". In: *Journal of physics*. 1722.1, p. 012104. ISSN: 1742-6588

This paper was recommended to be by peer feedback so I searched for it on Google Scholar by title.

The authors teach an algorithms and programming course and wanted to test the

flipped classroom and blended learning model on those 50 students. They used the "MUSIC® Model of Academic Motivation" questions to evaluate whether the teaching model was successful for students (Darmawan et al., 2021). Based on these questions, the students reported highest scores on the "usefulness" questions but lowest on the "success" questions (Darmawan et al., 2021).

I was very intrigued by the MUSIC® reflection questions. The questions that students reported highest on were "usefulness"; those questions gauge whether the course content was relevant and would help them in the future (Darmawan et al., 2021). However, they scored lowest on "success" questions which asked students if they felt that they had the tools they needed to do well in the course (Darmawan et al., 2021). To me, this reads that the flipped classroom was not very effective if students did not feel that they would be successful. However, there is no comparison here to past students' scores, so perhaps all of the students' answers increased overall. This list of questions looks interesting and I would like to implement them (or something similar) in my own project.

1.2.2 The Effectiveness and Experiences of Blended Learning Approaches to Computer Programming Education

Omer Deperlioglu and Utku Kose (2013). "The effectiveness and experiences of blended learning approaches to computer programming education". In: *Computer applications in engineering education*. 21.2, pp. 328–342. ISSN: 1061-3773

This paper was recommended to be by peer feedback so I searched for it on Google Scholar by title.

The authors taught a data structures and algorithms course to 100 students, where 50 were in the control group and 50 in the experimental group. The results were that 76% of the experimental group passed the class while only 40% of the control group passed. The control group learned in a face-to-face classroom setting while the experimental group learned via a blended learning model (some online resources, some face-to-face teaching) (Deperlioglu and Kose, 2013).

I am so glad to finally read a study that has a control group with such an incredible measurable difference. I wonder what the most effective piece is, whether it's just that students benefit from a variety of styles or if it's because students can study more effectively on their own time or something else entirely. It is also interesting that in the post-course survey students wrote comments that they

enjoyed learning in the blended model. This definitely seems like an interesting avenue to explore further (Deperlioglu and Kose, 2013).

1.2.3 *Blended Learning Experience in a Programming Language Course and the Effect of the Thinking Styles of the Students on Success and Motivation*

Mustafa Yagci (2016). "Blended Learning Experience in a Programming Language Course and the Effect of the Thinking Styles of the Students on Success and Motivation." In: *Turkish Online Journal of Educational Technology-TOJET* 15.4, pp. 32-45

This paper was recommended to be by peer feedback so I searched for it on Google Scholar by title.

The authors of this study divided 45 second-grade students up by their gender and learning style ("extroverted" versus "self-enclosed") and taught them programming in a blended learning style course (Yagci, 2016). They found that the pre and post test scores were higher when they used blended learning methods. Many of their tested hypotheses did not show a significant difference (differences between gender, etc.) (Yagci, 2016).

This was an interesting paper not so much for the actual research but rather for their very well written introduction. They give an overview on the research about students' different learning styles and how that can affect their success. I will look into papers that they referenced more during this assignment.

1.2.4 *Evaluation of Blended Learning Approach in Computer Engineering Education*

Tuncay Yigit et al. (2014). "Evaluation of Blended Learning Approach in Computer Engineering Education". In: *Procedia-social and behavioral sciences*. 141, pp. 807-812. ISSN: 1877-0428

This paper was recommended to be by peer feedback so I searched for it on Google Scholar by title.

In this study, the authors equally divided up 100 students in an algorithms course into two groups: a blended learning group and an in-person classroom group. According to the table, the blended learning group did approximately as well as the in-person group on assignments, but did worse on exams (approximately a 7 point average difference between the groups on both exams). The

study concludes that students benefited from this blended learning approach because the differences in scores were not huge but they saved time and money (Yigit et al., 2014).

This was an interesting paper because blended learning was not the huge improvement that others have seen in their studies. It does make me wonder what factors led to this. Perhaps the implementation of blended learning could improve in this course or perhaps the students in this group have a more hands-on learning style. I enjoyed reading an example where blended learning was not a magic catch-all solution to educational problems and I am curious why exam scores were lower for those students.

1.2.5 Interactive Git: A Web Application for Helping Users Understand Common Git Workflows

James Wang (2017). "Interactive Git: A Web Application for Helping Users Understand Common Git Workflows". PhD thesis. Mills College

A coworker shared with me an interesting git learning tool and I wanted to learn about similar tools, so I searched for "learn git branching interactive" on Google Scholar.

This is a Masters in Computer Science thesis paper about a tool the author created called "Interactive Git" (Wang, 2017). The author based his idea off of some other Git tutorials and wanted it to be a fully in-browser experience that simulated the terminal, file system, and Git. He implemented the project using only JavaScript (React) with no database. At the end, he showed the project to five people who were familiar with Git and they gave him his opinions on possible improvements (Wang, 2017).

I wanted to find a resource describing something similar to what I want to build this semester. Though there are definite differences (I will have no need for a simulated terminal or file system), the high-level goals of creating an interactive teaching tool with JavaScript (React) are the same. It seems from his description that it was a lot of work to create all of these parts and perhaps it would make sense to scale down as much as possible and then create a list of "nice to haves" if there is more time.

1.2.6 *Experiences of a Software Engineering Course based on Interactive Learning.*

Stephan Krusche, Nadine von Frankenberg, and Sami Afifi (2017). "Experiences of a Software Engineering Course based on Interactive Learning." In: *SEUH*, pp. 32-40

I found this paper in the list when searching for the previous paper, when I searched "learn git branching interactive" on Google Scholar.

The authors of this study teach a software engineering class that is comprised of hundreds of students (345 in 2014 and 272 in 2016). They implemented a new interactive learning curriculum in 2016 to see if that increased in-class participation, and they found that it did. Their new curriculum had five phases: theory (the professor gives a short lecture), example (the professor applies the lecture material), exercise (students complete a short assignment or quiz), solution (the professor goes over the solution), and reflection (the students discuss the exercise and lecture material) (Krusche, Frankenberg, and Afifi, 2017).

This is an interesting paper because it very specifically divides out the different phases of an interactive classroom. I would like to implement something similar in my own project and wonder how that translates well to a fully interactive online platform. For example, perhaps at the end of the exercise there is a discussion board where all students can post questions and responses. I would be interested to see if that can help students tease out common mistakes (for example, perhaps one student posts about a mistake they were making that they have now solved, and that can help another student).

1.2.7 *An Interactive Serious Game via Visualization of Real Life Scenarios to Learn Programming Concepts*

Sajana A, Kamal Bijlani, and R Jayakrishnan (2015). "An interactive serious game via visualization of real life scenarios to learn programming concepts". In: *Computing, Communication and Networking Technologies (ICCCNT), 2015 6th International Conference on*. 2015 6th International Conference on Computing, Communication and Networking Technologies (ICCCNT). IEEE, pp. 1-8. ISBN: 1-4799-7983-X

I found this paper by searching for "learn interactive visualization program-

ming" on Google Scholar.

In this study, the authors wanted to compare lecture-based learning and game-based learning for teaching introductory computer science topics. They equally divided 20 undergraduate students into two groups; one group watched normal lectures about basic concepts (variables and data types) while the second group played games that explained these concepts (A, Bijlani, and Jayakrishnan, 2015). At the end the students took an exam and filled out a reflective form on their experience. Students in the game group performed better on the exam and reported that the game was fun (A, Bijlani, and Jayakrishnan, 2015).

This was a small sample size but it is encouraging that students actually can learn from a game. I would be concerned that they would enjoy the game but not necessarily gain more information from it. The games themselves look like they probably took a lot of time to build up-front, however they can be reusable for many future semesters. The games might be a good supplement activity for students in addition to lectures to talk about more specifics.

1.2.8 Systematic Development of Dynamic Programming Algorithms Assisted by Interactive Visualization

J. Ángel Velázquez-Iturbide and Antonio Pérez-Carrasco (2016). "Systematic Development of Dynamic Programming Algorithms Assisted by Interactive Visualization". In: *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*. New York, NY, USA: Association for Computing Machinery, pp. 71–76. ISBN: 9781450342315. URL: <https://doi.org/10.1145/2899415.2899450>

I found this paper by searching for "learn interactive visualization programming" on Google Scholar.

The authors of this study teach an algorithms course that involves dynamic programming. They wanted to use SRec, a Java-based visualization of recursive calls, to help students learn. They divided the students into a control group and experimental group (difference was whether they used SRec or not) to refactor "redundant" algorithms into "efficient" algorithms (Velázquez-Iturbide and Pérez-Carrasco, 2016). The results were mixed; students in the experimental group had fewer errors and completed the assignment faster but they did not utilize SRec correctly so they were not as thorough (Velázquez-Iturbide and

Pérez-Carrasco, 2016).

This was a great paper to read because it is similar to something that I want to build (an interactive visualization of recursion) but it shows that even powerful visualizations are useless if the students do not use them correctly. Additionally, students rely too much on the visualization and do not think through the problem themselves as thoroughly. This reinforces the idea that there needs to be a mix of use of visualization and real application to reinforce concepts.

1.2.9 Using Visualization to Teach Novices Recursion

Wanda Dann, Stephen Cooper, and Randy Pausch (2001). "Using visualization to teach novices recursion". In: *Proceedings of the 6th annual conference on Innovation and technology in computer science education*, pp. 109–112

I found this by searching for "learn interactive visualization dynamic programming recursion" on Google Scholar.

The authors taught students recursion using Alice, "a 3-D interactive graphics programming environment" developed by Carnegie Mellon University (Dann, Cooper, and Pausch, 2001). Alice allowed students to write code and see in action two problems: a rabbit chasing a butterfly, and the Towers of Hanoi problem. There is no specific evidence (such as test scores) but students self-reported understanding recursion better after using this tool (Dann, Cooper, and Pausch, 2001).

This seems like an interesting version of showing students the Towers of Hanoi problem, and also seems like a much more interactive version because students are running the code rather than watching other students act it out. However, there is no hard evidence as to whether or not students understood recursion better after using the tool. Also, I wonder how long it takes to get familiar with Alice enough to set these problems up for students.

1.2.10 Online Python Tutor: Embeddable Web-Based Program Visualization for CS Education

Philip J Guo (2013). "Online python tutor: embeddable web-based program visualization for cs education". In: *Proceeding of the 44th ACM technical symposium on Computer science education*, pp. 579–584

I found this by searching for "teach using d3 recursion" on Google Scholar.

Though this was not what I was expecting, I have heard of Python Tutor before so I was interested in reading more about it.

The authors developed a website called Online Python Tutor (which now also accepts other languages such as JavaScript). The website takes in input code from the user and then runs the code line-by-line as the interpreter, updating the visualization with variable information as it goes. This helps students see the recursive call stack and trace the values more easily (Guo, 2013).

This website is pretty incredible and I had never used it before. I tried it with a recent problem we gave our students in JavaScript and I love how it traces the code so easily for students to follow. This is extremely similar to what I was imagining I would build in this course, so it seems that what I wanted to build may not be necessary. However, it is good to see this as a starting point and think about if there is anything that could be clearer.

1.2.11 A Feedback-Oriented Platform for Deliberate Programming Practice

Mario Sánchez and Pedro Salazar (2020). "A feedback-oriented platform for deliberate programming practice". In: *Proceedings of the 2020 ACM Conference on Innovation and Technology in Computer Science Education*, pp. 531–532

I found this by searching for "exercism" on Google Scholar. Exercism is a website that teaches different programming languages and I was interested to see if papers referenced it or had done research around it.

Professors at a university in Colombia who teach over 900 students per semester in their introductory computer science course needed a better tool to grade students and give them valuable feedback. They developed Senecode, which is a web platform that tags different algorithmic problems by difficulty and problem type and then runs various specific tests with descriptive errors. They allowed students to re-submit their code as many times as necessary to get the right answers. They found that the majority of students were able to understand the errors and eventually get the correct answer (Sánchez and Salazar, 2020).

This honestly was not as ground-breaking as I had hoped, considering that there are many websites that already do this (LeetCode, etc.). I was hoping for a new approach to delivering feedback, whether that was accumulating common issues as "hints" to the student or something else. However, I appreciate that the students were able to eventually find the answer because they were given enough

guidance to make progress. Also, I would have liked to see some sort of reflective form from students to see if they felt Senecode was helpful to them.

1.2.12 Automated Feedback Generation for Introductory Programming Assignments

Rishabh Singh, Sumit Gulwani, and Armando Solar-Lezama (2013). “Automated feedback generation for introductory programming assignments”. In: *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*, pp. 15–26

I found this paper by searching for "feedback programming" on Google Scholar.

The authors of this paper created a tool that can automatically read a student's input code, compare that code against approximately 10^7 potential fixes, and then respond back with the fixes the student needs to make. They write the potential fixes in EML (Error Model Language) and the different combinations of those rules is what the computer tries to use to fix the student code. They found that their program can fix approximately 65% of incorrect student submissions (Singh, Gulwani, and Solar-Lezama, 2013).

This tool is pretty incredible and I am very impressed that they were able to build something this dynamic. The actual implementation was way over my head and I do not think I would be able to implement anything similar (especially in just a semester). However, this is an inspiring paper and I hope that this tool becomes more widespread in the future.

1.2.13 Performance and Use Evaluation of an Electronic Book for Introductory Python Programming

Christine Alvarado et al. (2012). “Performance and use evaluation of an electronic book for introductory Python programming”. In: *Technical Report GT-IC-12-02*

This paper was referenced in an earlier paper that I read and that paper claimed that students who used embedded visualizations had better exam scores, so I wanted to read more (Guo, 2013).

Miller and Ranum, who are the authors of an e-book for learning computer science, combined their e-book with videos, a visualization tool, and an area for students to write and test code. The authors of this paper wanted to study

Miller and Ranum's students to see how the students' interactions with these materials affected their grades on exams. They found that students who used the visualization tool outside of class on their own tended to have higher exam scores (Alvarado et al., 2012).

The authors point out that this does not necessarily mean cause and effect, perhaps the students who scored better on the exams were also more interested in the material or more motivated to study outside of class regardless of the e-book. I do appreciate that according to a reflection form students liked the resources available to them. The variety of types of resources seems to be a key feature of many positive studies about teaching programming.

1.2.14 *Automatic Grading and Feedback Using Program Repair for Introductory Programming Courses*

Sagar Parihar et al. (2017). "Automatic grading and feedback using program repair for introductory programming courses". In: *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education*, pp. 92–97

I found this paper in the references of a previous paper, (Sánchez and Salazar, 2020).

The authors of this paper were struggling with grading many students' assignments so they created GradeIT, a program that gave meaningful feedback to students' code and repaired code that had common syntax errors. This saved time for teaching assistants so they did not have to hand-grade, it allowed for more consistent grading practices, and students benefited from getting immediate feedback. By using GradeIT, the number of errored submissions was reduced by 62% for exams (Parihar et al., 2017).

This is a pretty interesting system and I am glad they were able to find the majority of coding errors automatically. I think that this is more relevant for strongly typed languages like Java where some errors (such as missing semicolons) are very common and easy to spot. However, I do like that they were able to immediately give the student another chance to submit rather than give them a bad grade due to a minor compilation error. In my future project, I hope to gather and display similar common feedback for students.

1.2.15 *Facilitating Course Assessment with a Competitive Programming Platform*

Daniel Coore and Daniel Fokum (2019). "Facilitating Course Assessment with a Competitive Programming Platform". In: *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. SIGCSE '19. Minneapolis, MN, USA: Association for Computing Machinery, pp. 449–455. ISBN: 9781450358903. DOI: [10.1145/3287324.3287511](https://doi.org/10.1145/3287324.3287511). URL: <https://doi.org/10.1145/3287324.3287511>

I found this paper in the references of a previous paper, (Sánchez and Salazar, 2020).

The authors of this study wanted to assess their undergraduate students quickly and give them immediate feedback while also making the class more fun by "gamifying" it (Coore and Fokum, 2019). They decided to include HackerRank contests (programming tests) to the list of assignments for the class. They were not able to determine that exam scores increased due to the contests and the students' feelings about the frequent contests were mixed (Coore and Fokum, 2019).

I assumed that the HackerRank contests would be fun for students and would encourage them to practice more. However, it seems that it did not help the students in their overall understanding. I wonder if there is a way the contests could be tweaked so that they would be beneficial, and I also wonder if there were other factors at play (perhaps they struggled with the style of HackerRank problems, for example).

1.2.16 *The Role of Deliberate Practice in the Acquisition of Expert Performance*

K Anders Ericsson, Ralf T Krampe, and Clemens Tesch-Römer (1993). "The role of deliberate practice in the acquisition of expert performance." In: *Psychological review* 100.3, p. 363

I found this paper in the references of a previous paper, (Sánchez and Salazar, 2020).

The authors define deliberate practice as "practice activities that the individual can engage in between meetings with the teacher", thus distinguishing them from work (such as performances) and play (activities that are for enjoyment but not betterment) (Ericsson, Krampe, and Tesch-Römer, 1993). The researchers

asked music professors to identify good and great violinists and then asked those violinists and the teachers to estimate how much time they spent doing various activities. All of them ranked practice as being the most relevant activity for improving their skill. The authors do comparable studies with other elite groups such as athletes; deliberate practice is a key aspect for all of these elite performers (Ericsson, Krampe, and Tesch-Römer, 1993).

This is not necessarily a groundbreaking study because it is common sense that an Olympic athlete is not born overnight. However, it is good to put into context with students that the best way for them to learn something is to actually practice. While many of these papers emphasize a new technique for teaching or giving feedback, at the root of all of these is the desire for students to practice more often and more effectively.

1.3 Synthesis

One of the themes that I explored this week in particular was on effective feedback. It seemed to be a very common problem with professors teaching computer science that they wanted to give immediate helpful feedback but lacked the time and resources to do so. One solution was to create tests with descriptive errors so the students could run their code and get some information on the effectiveness of their solution (Sánchez and Salazar, 2020). Another research group went a step further and made a tool that would fix students' code for them (basic syntax errors and other common mistakes) (Singh, Gulwani, and Solar-Lezama, 2013), (Parihar et al., 2017).

Another theme I explored was on blended learning and flipped classrooms. Some studies found that blending learning was incredible effective and led to much higher pass rates and test scores (Deperlioglu and Kose, 2013) (Yagci, 2016). Other studies did not have such success; one found that students reflected the learning style was useful but they did not feel they had the tools they needed to succeed (Darmawan et al., 2021), and another study found that students actually did worse on exams (Yigit et al., 2014). It was interesting to see papers on the same topic but different conclusions; it makes you wonder if there are some other factors at play or if blended learning has to be done a certain way in order to be successful.

In addition, I looked into many different types of visualizations for teaching computer science topics (Wang, 2017), (Guo, 2013). Students were successful

when they learned about concepts from games (A, Bijlani, and Jayakrishnan, 2015), (Dann, Cooper, and Pausch, 2001). However, sometimes the visualization was also a crutch where students were not as thorough in their understanding (Velázquez-Iturbide and Pérez-Carrasco, 2016). Additionally, it was unclear in some cases if the visualization helped students or if students who used the visualization were already more comfortable with the material (Alvarado et al., 2012).

Finally, I looked into the role of deliberate practice in the learning process. Elite musicians reflected that practice was the most important part of their process (Ericsson, Krampe, and Tesch-Römer, 1993). However, the practice has to be comprised of the correct activities. In another study, assigning students to do HackerRank style competitions did not improve their final scores (Coore and Fokum, 2019).

1.4 Reflection

Every week has been incrementally easier, though I would definitely not consider it an easy task for me. I am extremely grateful that my mentor and peer reviewers have given my good advice on what topics to look into next.

This week, I wanted to learn more about blended learning and flipped classrooms, using visualizations to teach programming concepts, and how to give effective feedback. These three goals made it a lot easier to focus on which papers to look for next. In previous assignments I had too many possible avenues so it was difficult to focus.

Going forward, I would like to learn more about implementation of these tools. Many of the papers I read discussed tools at a high level but it was difficult to understand how they were built. I did find a good repository for a Git visualization, so I may use that as a guide for how to structure my own visualization. I am also very open to any suggestions readers may have because it does still feel like an overwhelming project to break down into smaller pieces.

1.5 Planning

I am interested in developing an application that combines different resources (videos, visualizations, etc.) for learning recursion. For each resource, I would like to develop a pre test and post test to evaluate if the resource helped the student understand. If there is time, I would like to also have a bank of test

questions that are ranked from easiest to hardest with automated testing so that students can evaluate their progress. Finally, I would like to have a discussion board so that students can post their own resources or questions and answers.

If there was unlimited time I would branch this out to other difficult problem types (various data structures, etc.) but with the time constraint I will stick with recursion. My current idea is that I will build out the basics so that more resources and practice questions can be added easily at a later time.

As for implementation details, I have the most experience making full stack web apps with React on the front end and either Node/Express or Python/Django on the back end, so that will probably be my tech stack.

2 ACTIVITY

2.1 Problem Statement

2.1.1 Background Information

Recursion is a problem solving strategy in computer programming where the function calls itself. For this strategy to work, the problem must be divisible into smaller problems that have the same structure. For example, in a tree data structure, each new node of the tree can also be considered the parent of its own tree. Therefore, the entire tree can be traversed recursively.

This is a common problem solving strategy, especially for job interview questions in software engineering. However, it is very difficult for novice programmers to understand. It is also a difficult topic to teach because it is not easy to follow the many recursive calls and keep track of all of the values.

2.1.2 General Problem Statement

The general problem statement is that it is difficult to teach recursion.

2.1.3 Scholarly Support

Over the course of the past few assignments, I have read many papers that indicate that recursion is difficult to teach for professors at universities around the world.

At Carnegie Mellon University they developed ChiQat-Tutor to allow students to try several strategies for learning recursion (AlZoubi et al., 2015). At some

universities they used games to teach students recursion (Lee et al., 2014) (Chaffin et al., 2009). Other universities used hands-on approaches like the Towers of Hanoi game (Benander and Benander, 2008). Others taught recursion first (before arrays even) so that it seemed easy in comparison (Bruce, Danyluk, and Murtagh, 2005). Several universities have taught recursion using graphs and tree diagrams (Hsin, 2008) (Kruse, 1982). Another research group had students use pen and paper to follow the order of execution in recursive problems (Tunnell Wilson, Fisler, and Krishnamurthi, 2018). Using visual fractals was also a common strategy (Elenbogen and O'Kennon, 1988) (Gordon, 2006). Finally, some teachers chose to use mathematical induction to explain recursion (Ford, 1984) (Wu, Dale, and Bethel, 1998).

A common theme between all of these papers is that teachers are trying to find the best way to teach recursion because it is such a difficult topic to teach novice programmers.

2.1.4 *Specific Problem Statement*

The more specific problem statement is that it is difficult to teach adult novice programmers how to solve recursive functions. There are many potential strategies and it is not clear which strategies are the most effective to the most number of students.

2.1.5 *Closing Commentary*

We need bright students from diverse backgrounds to learn computer science so that those students can use their nuanced perspectives to solve difficult problems in the world. Unfortunately, recursion is difficult to teach so students tend to be hard on themselves and think that they lack the skills to be successful in this field. If we can pinpoint the best ways to teach recursion to students, then we can also better teach other programming strategies, and computer science can be a more inclusive field. If we cannot solve this problem, I think that we will miss out on many students who were perfectly capable of being excellent computer scientists but were made to feel that they were incapable.

2.2 Research Questions

The research question I would like to answer is "What is the most effective way to teach adult novice programmers recursion?" That question can be divided up into three sub questions, as explained below.

2.2.1 *What are some effective ways of teaching recursion?*

Complex: There are many ways to teach recursion including mathematical proofs, visualizations using code, games, hands-on interactive exercises, code insertion, and more. However, that does not mean that all of them are effective and particularly for adult novice programmers. This question is complex because there are many possible answers.

Arguable: However, they are arguable because they can be tested. For example, you could divide a classroom of students into several groups where each group is taught in a different way, and then they are all given the same exam on recursion at the end. Alternatively, students could be shown different resources but given a pre and post test to see how that resource affected their overall understanding.

2.2.2 *To what extent does the combination of resources impact students' understanding of recursion?*

Complex: It is possible that there is no "one size fits all" approach for teaching recursion, and that a few resources combined give students a better holistic understanding. It is also possible that there is one extremely effective strategy for teaching recursion and students need to focus on that strategy in multiple sessions rather than waste their time with other strategies. There are many possible combinations of resources and the order can also matter (perhaps one resource is only effective after being shown others first).

Arguable: This can be tested by dividing students into different groups with a variety of combinations. For example, let's say there are two types of resources: videos and exercises. The students can be divided into groups of all the combinations: one only sees one video, one only does one exercise, one sees a video then does an exercise, one does an exercise then watches a video, one sees two videos, and one completes two exercises. At the end, all four groups take the same exam and then the results are compared.

2.2.3 *To what extent do the different types of recursive problems impact students' understanding of recursion?*

Complex: There are many types of recursive problems such as tail, binary, and mutual. Some of those problems may be easier to learn than others, and perhaps

learning one type first can make others easier to learn later on. For example, some of my students find inner recursive functions to be conceptually easier for them to understand than recursively calling the whole function. It is possible that once they have learned one type they can learn the other types easier, but it is also possible that they get mentally boxed into the one type and have trouble branching out into new types. It would be interesting to see if there is a good introductory type or if a particular type should always be learned last because it is so complex.

Arguable: Once again, we could test this by using students and exam scores. Students can be shown examples and given assignments where they are exposed to problems from all categories. Then on the exam they can be given a few questions from each category. The exam scores can help researchers identify if certain types are easier for students to understand than others. Then future studies can be run where students are only shown one type and then later tested on another type to see if their knowledge is easily transferable to other types.

3 REFERENCES

- [1] A, Sajana, Bijlani, Kamal, and Jayakrishnan, R (2015). "An interactive serious game via visualization of real life scenarios to learn programming concepts". In: *Computing, Communication and Networking Technologies (ICC-CNT), 2015 6th International Conference on*. 2015 6th International Conference on Computing, Communication and Networking Technologies (ICC-CNT). IEEE, pp. 1–8. ISBN: 1-4799-7983-X.
- [2] Alvarado, Christine, Morrison, Briana B, Ericson, Barbara, Guzdial, Mark, Miller, Brad, and Ranum, David L (2012). "Performance and use evaluation of an electronic book for introductory Python programming". In: *Technical Report GT-IC-12-02*.
- [3] AlZoubi, Omar, Fossati, Davide, Di Eugenio, Barbara, Green, Nick, Alizadeh, Mehrdad, and Harsley, Rachel (2015). "A Hybrid Model for Teaching Recursion". In: *Proceedings of the 16th Annual Conference on Information Technology Education*. SIGITE '15. Chicago, Illinois, USA: Association for Computing Machinery, pp. 65–70. ISBN: 9781450338356. DOI: [10.1145/2808006.2808030](https://doi.org/10.1145/2808006.2808030). URL: <https://doi.org/10.1145/2808006.2808030>.
- [4] Benander, Alan C. and Benander, Barbara A. (Winter 2008). "Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Tow-

- ers of Hanoi”. English. In: *Journal of Information Systems Education* 19.4. Copyright - Copyright EDSIG Winter 2008; Document feature - ; Tables; Diagrams; Graphs; Last updated - 2020-11-17, pp. 455-467. URL: <https://go.openathens.net/redirector/gatech.edu?url=https://search-proquest-com.eu1.proxy.openathens.net/scholarly-journals/student-monks-teaching-recursion-is-cs/docview/200157798/se-2?accountid=11107>.
- [5] Bruce, Kim B, Danyluk, Andrea, and Murtagh, Thomas (2005). “Why structural recursion should be taught before arrays in CS 1”. In: *SIGCSE bulletin*. 37.1, pp. 246-250. ISSN: 0097-8418.
 - [6] Chaffin, Amanda, Doran, Katelyn, Hicks, Drew, and Barnes, Tiffany (2009). “Experimental Evaluation of Teaching Recursion in a Video Game”. In: *Proceedings of the 2009 ACM SIGGRAPH Symposium on Video Games*. New York, NY, USA: Association for Computing Machinery, pp. 79-86. ISBN: 9781605585147. URL: <https://doi.org/10.1145/1581073.1581086>.
 - [7] Coore, Daniel and Fokum, Daniel (2019). “Facilitating Course Assessment with a Competitive Programming Platform”. In: *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. SIGCSE '19. Minneapolis, MN, USA: Association for Computing Machinery, pp. 449-455. ISBN: 9781450358903. DOI: [10.1145/3287324.3287511](https://doi.org/10.1145/3287324.3287511). URL: <https://doi.org/10.1145/3287324.3287511>.
 - [8] Dann, Wanda, Cooper, Stephen, and Pausch, Randy (2001). “Using visualization to teach novices recursion”. In: *Proceedings of the 6th annual conference on Innovation and technology in computer science education*, pp. 109-112.
 - [9] Darmawan, ID MBA, Wirastuti, NMAED, Nilakusmawati, DPE, Raharja, MA, et al. (2021). “The effectiveness of the blended learning approach in algorithm and programming courses”. In: *Journal of physics*. 1722.1, p. 012104. ISSN: 1742-6588.
 - [10] Deperlioglu, Omer and Kose, Utku (2013). “The effectiveness and experiences of blended learning approaches to computer programming education”. In: *Computer applications in engineering education*. 21.2, pp. 328-342. ISSN: 1061-3773.
 - [11] Elenbogen, Bruce S. and O’Kennon, Martha R. (Feb. 1988). “Teaching Recursion Using Fractals in Prolog”. In: *SIGCSE Bull.* 20.1, pp. 263-266. ISSN:

- 0097-8418. DOI: [10.1145/52965.53029](https://doi.org/10.1145/52965.53029). URL: <https://doi.org/10.1145/52965.53029>.
- [12] Ericsson, K Anders, Krampe, Ralf T, and Tesch-Römer, Clemens (1993). "The role of deliberate practice in the acquisition of expert performance." In: *Psychological review* 100.3, p. 363.
 - [13] Ford, Gary (Jan. 1984). "An Implementation-Independent Approach to Teaching Recursion". In: *SIGCSE Bull.* 16.1, pp. 213–216. ISSN: 0097-8418. DOI: [10.1145/952980.808655](https://doi.org/10.1145/952980.808655). URL: <https://doi.org/10.1145/952980.808655>.
 - [14] Gordon, Aaron (Oct. 2006). "Teaching Recursion Using Recursively-Generated Geometric Designs". In: *J. Comput. Sci. Coll.* 22.1, pp. 124–130. ISSN: 1937-4771.
 - [15] Guo, Philip J (2013). "Online python tutor: embeddable web-based program visualization for cs education". In: *Proceeding of the 44th ACM technical symposium on Computer science education*, pp. 579–584.
 - [16] Hsin, Wen-Jung (Apr. 2008). "Teaching Recursion Using Recursion Graphs". In: 23.4, pp. 217–222. ISSN: 1937-4771.
 - [17] Krusche, Stephan, Frankenberg, Nadine von, and Afifi, Sami (2017). "Experiences of a Software Engineering Course based on Interactive Learning." In: *SEUH*, pp. 32–40.
 - [18] Kruse, Robert L. (Feb. 1982). "On Teaching Recursion". In: *SIGCSE Bull.* 14.1, pp. 92–96. ISSN: 0097-8418. DOI: [10.1145/953051.801346](https://doi.org/10.1145/953051.801346). URL: <https://doi.org/10.1145/953051.801346>.
 - [19] Lee, Elynn, Shan, Victoria, Beth, Bradley, and Lin, Calvin (2014). "A Structured Approach to Teaching Recursion Using Cargo-Bot". In: *Proceedings of the Tenth Annual Conference on International Computing Education Research*. ICER '14. Glasgow, Scotland, United Kingdom: Association for Computing Machinery, pp. 59–66. ISBN: 9781450327558. DOI: [10.1145/2632320.2632356](https://doi.org/10.1145/2632320.2632356). URL: <https://doi.org/10.1145/2632320.2632356>.
 - [20] Parihar, Sagar, Dadachanji, Ziyaan, Singh, Praveen Kumar, Das, Rajdeep, Karkare, Amey, and Bhattacharya, Arnab (2017). "Automatic grading and feedback using program repair for introductory programming courses". In: *Proceedings of the 2017 ACM Conference on Innovation and Technology in Computer Science Education*, pp. 92–97.
 - [21] Sánchez, Mario and Salazar, Pedro (2020). "A feedback-oriented platform for deliberate programming practice". In: *Proceedings of the 2020 ACM Con-*

- ference on Innovation and Technology in Computer Science Education*, pp. 531–532.
- [22] Singh, Rishabh, Gulwani, Sumit, and Solar-Lezama, Armando (2013). “Automated feedback generation for introductory programming assignments”. In: *Proceedings of the 34th ACM SIGPLAN conference on Programming language design and implementation*, pp. 15–26.
 - [23] Tunnell Wilson, Preston, Fisler, Kathi, and Krishnamurthi, Shriram (2018). “Evaluating the Tracing of Recursion in the Substitution Notional Machine”. In: *Proceedings of the 49th ACM Technical Symposium on Computer Science Education. SIGCSE ’18*. Baltimore, Maryland, USA: Association for Computing Machinery, pp. 1023–1028. ISBN: 9781450351034. DOI: [10 . 1145 / 3159450 . 3159479](https://doi.org/10.1145/3159450.3159479). URL: <https://doi.org/10.1145/3159450.3159479>.
 - [24] Velázquez-Iturbide, J. Ángel and Pérez-Carrasco, Antonio (2016). “Systematic Development of Dynamic Programming Algorithms Assisted by Interactive Visualization”. In: *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*. New York, NY, USA: Association for Computing Machinery, pp. 71–76. ISBN: 9781450342315. URL: <https://doi.org/10.1145/2899415.2899450>.
 - [25] Wang, James (2017). “Interactive Git: A Web Application for Helping Users Understand Common Git Workflows”. PhD thesis. Mills College.
 - [26] Wu, Cheng-Chih, Dale, Nell B., and Bethel, Lowell J. (Mar. 1998). “Conceptual Models and Cognitive Learning Styles in Teaching Recursion”. In: *SIGCSE Bull.* 30.1, pp. 292–296. ISSN: 0097-8418. DOI: [10 . 1145 / 274790 . 274315](https://doi.org/10.1145/274790.274315). URL: <https://doi.org/10.1145/274790.274315>.
 - [27] Yagci, Mustafa (2016). “Blended Learning Experience in a Programming Language Course and the Effect of the Thinking Styles of the Students on Success and Motivation.” In: *Turkish Online Journal of Educational Technology-TOJET* 15.4, pp. 32–45.
 - [28] Yigit, Tuncay, Koyun, Arif, Yuksel, Asim Sinan, and Cankaya, Ibrahim Arda (2014). “Evaluation of Blended Learning Approach in Computer Engineering Education”. In: *Procedia–social and behavioral sciences*. 141, pp. 807–812. ISSN: 1877-0428.